

Volume 49

Sentinel Python Runtime Architecture (SPRA)

Status: Core Engineering Specification

Priority: Highest

Vision

Project Sentinel shall be implemented as a collection of independent Python packages running on one common runtime.

No monolithic application.

No spaghetti dependencies.

No circular imports.

No hidden coupling.

Runtime Philosophy

The runtime should behave exactly like the architecture.

Objects

↓

Events

↓

Knowledge

↓

Evidence

↓

Trust

↓

Missions

↓

Workflows

↓

Decisions

↓

Presentation

Every layer only depends on the layers below it.

Repository Layout

project-sentinel/

Every package has a clear responsibility.

Kernel First

The kernel starts before everything else.

Startup sequence:

Kernel

Nothing bypasses the kernel.

Python Philosophy

We deliberately stay close to standard Python.

Primary principles:

- Readability.
 - Type safety.
 - Async-first where appropriate.
 - Dependency injection.
 - Explicit interfaces.
 - Comprehensive testing.
-

Async Architecture

Long-running tasks should not block the platform.

Examples:

- Evidence collection
- Worker coordination
- Cloud synchronization

- Mobile fleet verification
- Report generation

Async execution keeps the user interface responsive.

Service Boundaries

Every engine exposes documented interfaces.

Example:

Trust Engine

Other engines never manipulate internal state directly.

Event Contracts

Every event follows a versioned schema.

Event

Schema evolution is managed explicitly.

Object Contracts

Every object implements common capabilities.

Required methods:

- create()
- update()
- verify()
- archive()
- relate()
- snapshot()
- history()

Consistency simplifies development.

Runtime Configuration

Configuration is layered.

Priority:

Defaults

Overrides remain predictable.

Plugin Loader

The runtime discovers plugins through manifests.

Validation includes:

- Signature.
- API version.
- Required permissions.
- Dependencies.
- Compatibility.

Plugins load dynamically without restarting the platform whenever feasible.

Dependency Management

Rules:

- No cyclic dependencies.
- Kernel depends on nothing above it.
- Security engines depend on kernel APIs only.
- UI depends on public interfaces.

Architecture is enforced automatically.

Error Handling

Every error becomes an event.

Example:

Collector Failed

Nothing disappears silently.

Logging Philosophy

Logs are not debugging output.

Logs become structured operational records.

Every entry includes:

- Correlation ID.
- Object ID.
- Mission ID.
- Event ID.
- Severity.
- Timestamp.

Logs support both operations and development.

Observability

The runtime continuously measures itself.

Examples:

- Startup time.
- Memory usage.
- CPU utilization.
- Worker queue depth.
- Event throughput.
- Plugin health.
- API latency.

Project Sentinel monitors Project Sentinel.

Runtime Security

The platform protects itself.

Measures include:

- Signed plugins.
- Configuration validation.
- Secret separation.
- Least-privilege execution.
- Integrity verification.
- Secure defaults.
- Audit logging.

Security is part of the runtime, not an optional add-on.

Testing Strategy

Every package includes:

- Unit tests.
- Integration tests.
- Contract tests.
- Performance tests.
- Security tests.

No package ships without automated verification.

Documentation Strategy

Documentation is generated from the codebase where practical.

Every package includes:

- Purpose.
- Architecture.
- Public interfaces.
- Examples.
- Version history.
- ADR references.

The documentation evolves with the implementation.

NEW CORE ENGINE

Runtime Health Engine

Continuously evaluates:

- Service availability.
- Worker status.
- Queue health.
- Plugin stability.
- Memory pressure.
- Event processing.

The platform can identify issues in itself before users notice them.

NEW CORE ENGINE

Runtime Upgrade Engine

Supports safe upgrades by:

- Verifying compatibility.
- Applying schema migrations.
- Validating plugins.
- Rolling back failed upgrades when possible.
- Recording every change.

This enables predictable long-term maintenance.

NEW CORE ENGINE

Configuration Validation Engine

Every configuration change is validated before activation.

Checks include:

- Syntax.
- Required fields.
- Dependency consistency.
- Policy conflicts.
- Version compatibility.

Invalid configurations never become active.

NEW IDEA

Sentinel Development Mode

A special runtime profile for developers.

Features:

- Live event inspection.
- Object explorer.
- Event replay.
- Mock evidence generators.
- Plugin debugger.
- Performance profiling.
- Architecture visualization.

Developers can understand the system while building it.

NEW IDEA

Sentinel Architecture Validator

One of the ideas I believe will keep the project maintainable.

The validator continuously checks:

- Forbidden dependencies.
- Layer violations.
- API misuse.
- Missing documentation.
- Missing tests.
- ADR compliance.

The architecture becomes enforceable rather than aspirational.

NEW IDEA

Runtime Time Machine

Every runtime change is versioned.

Developers can inspect:

- Configuration at a given date.
- Active plugins.
- Loaded services.
- Event schemas.
- Object models.

This simplifies debugging and audits.

PS-ADR-0044

Stable Runtime, Replaceable Engines

Status: Accepted

Decision

Project Sentinel shall maintain a stable runtime kernel while allowing individual engines, plugins, and integrations to evolve independently through documented interfaces and versioned contracts.

Rationale

A stable runtime minimizes long-term maintenance costs, reduces upgrade risk, and enables decades of incremental evolution without requiring major architectural redesign.

NEW LONG-TERM VISION

Sentinel Engineering Platform

At this stage, Project Sentinel is not just a cybersecurity product.

It is also an engineering platform.

Every component:

- can be tested,
- can be replaced,
- can be versioned,
- can be documented,
- can be observed,
- can be validated.

That is how we avoid the legacy problems you wanted to eliminate from the very beginning.

A strategic proposal before implementation

I think we should adopt one more architectural principle that will guide every line of code we write:

If a new developer cannot understand a module after reading its documentation and code for one hour, the module is too complex and should be redesigned.

This principle reinforces our shared goal that Project Sentinel should remain **powerful, elegant, and understandable**, regardless of how large it becomes.

Progress Update

With Volume 49 completed, I estimate the blueprint is approximately **82-85% complete**.

The remaining volumes will focus on:

- **Volume 50 - Data Protection & Privacy Engine**
- **Volume 51 - Frontend & UX Architecture**
- **Volume 52 - Deployment & Operations**
- **Volume 53 - Engineering Standards & Quality**
- **Volume 54 - Enterprise Scaling & High Availability**
- **Volume 55 - Implementation Roadmap (Phase 1 → Production)**

Once those are finished, we will have a complete engineering blueprint that can guide implementation in Python from the kernel outward, allowing us to build Project Sentinel incrementally while preserving the clean, evidence-driven architecture we've established. I believe that disciplined approach is what gives the project its best chance of remaining both innovative and maintainable over the long term.

====[USER]====

good and continue

====[ASSISTANT]====

Excellent. Next we design one of the most important engines for banks, crypto companies, governments, healthcare, and private users.

PROJECT SENTINEL BLUEPRINT

Volume 50